

STAT 651 - Final Project

Dallin Mason, Andrew Stradling, RJ Cass

Introduction

It is generally accepted that certain factors influence your expected income. Some of these factors include race, gender, level of education, etc. We want to use the 1994 US Census data available on Kaggle to determine how certain demographic factors affect a person's likelihood of earning more than \$50k annually.

Data

The data are extracted from the 1994 United States census. This data was preprocessed and downloaded on the Kaggle website, which obtained the data from the UC Irvine machine learning repository. Each row in the dataset is an individual, where the response (y_i) is a binary variable indicating whether the individual's annual income exceeded \$50,000, and a series of demographic factors. Of the available factors, we will use: Age, Hours worked per week, Education Level (highest education achieved), Marital Status, Race, and Gender. The dataset does have other factors available, such as Occupation, but we specifically want to examine the effect of these demographic values not accounting for specific jobs/roles.

The Education factor has many levels (16), so we decided to bucket these levels into similar groups of: Elementary/Middle, HighSchool, Undergraduate, Graduate. We also condensed MaritalStatus into just whether or not the person is married. Summary statistics of each variable are provided in Table 1 and Table 2.

Table 1

Variable	Mean	SD	Q25	Q75
age	38.62	13.73	28	48
hours-per-week	40.41	12.4	40	45

Table 2

Variable	Value	Count (Percent)
education	Elementary	767 (1.7%)
education	Graduate	2928 (6.7%)

Table 2

Variable	Value	Count (Percent)
education	HighSchool	19227 (43.7%)
education	Undergraduate	21035 (47.9%)
race	Amer-Indian-Eskimo	421 (1%)
race	Asian-Pac-Islander	1373 (3.1%)
race	Black	4218 (9.6%)
race	Other	373 (0.8%)
race	White	37572 (85.5%)
gender	Female	14557 (33.1%)
gender	Male	29400 (66.9%)
marital	Married	20748 (47.2%)
marital	Single	23209 (52.8%)

Model

We plan to fit a logistic regression model, described as follows:

$$\begin{aligned}
y_i &\sim \text{Bernoulli}(p_i) \\
\text{logit}(p_i) = \eta_i &= \beta_0 + \beta_1 \text{Age} + \beta_2 \text{HoursPerWeek} + \sum_{j=1}^3 \beta_{3,j} \text{Education}_j + \beta_4 \text{MaritalStatus} \\
&+ \sum_{k=1}^4 \beta_{5,k} \text{Race}_k + \beta_6 \text{GenderMale}
\end{aligned}$$

In this model, the Education, MaritalStatus, Race, and Gender variables are indicator variables which indicate whether or not a certain person falls into that category. Age and HoursPerWeek are continuous variables so the corresponding β 's represent the slope of the effect of those factors.

Priors

For each set of β 's we picked a normal prior. For the continuous variables (Age and HoursPerWeek), because we expect these β 's to be positive (we expect older people to have a higher likelihood of earning more than \$50k), we picked a slightly-positive mean with a smaller standard deviation so it is more weighted towards positive values. For the other β 's we picked the mean to be 0, with wider standard deviations as less-informative priors. The specific values used are provided in Table 3.

Table 3: Chosen priors for the analysis

Variable	β	Prior	Chosen Values
Intercept	0	$N(0, \sigma^2)$	$\sigma^2 = 10$

Variable	β	Prior	Chosen Values
Age	1	$N(\pi, \gamma^2)$	$\pi = 0.05, \gamma = 0.5$
HoursPerWeek	2	$N(\kappa, \psi^2)$	$\kappa = 0.03, \psi = 0.5$
Education	3-5	$N(\theta, \nu^2)$	$\theta = 0, \nu = 0.7$
MaritalStatus	6	$N(\mu, \tau^2)$	$\mu = 0, \tau = 0.7$
Race	7-10	$N(\lambda, \rho^2)$	$\lambda = 0, \rho = 0.7$
Gender	11	$N(\phi, \epsilon^2)$	$\phi = 0, \epsilon = 0.7$

We also implemented a hierarchical prior on some of the race categories, where we said the means of the non-white race categories have a shared $N(\lambda_s, \rho_s^2), \lambda_s = 0, \rho_s = 1$ prior. This will enable them to ‘share strength’ as we believe these race categories should behave similarly.

Sampler Performance

To measure our model performance, we generated a set of fake data using ‘true’ parameters based on our priors. We ran each sampler on this fake data to see if each sampler could capture the true values. As shown in Figure 1, both samplers are able to capture almost all of the true parameters within their posterior 95% confidence intervals. This gives us confidence that the samplers are performing the sampling expected and correctly exploring the required space for each parameter.

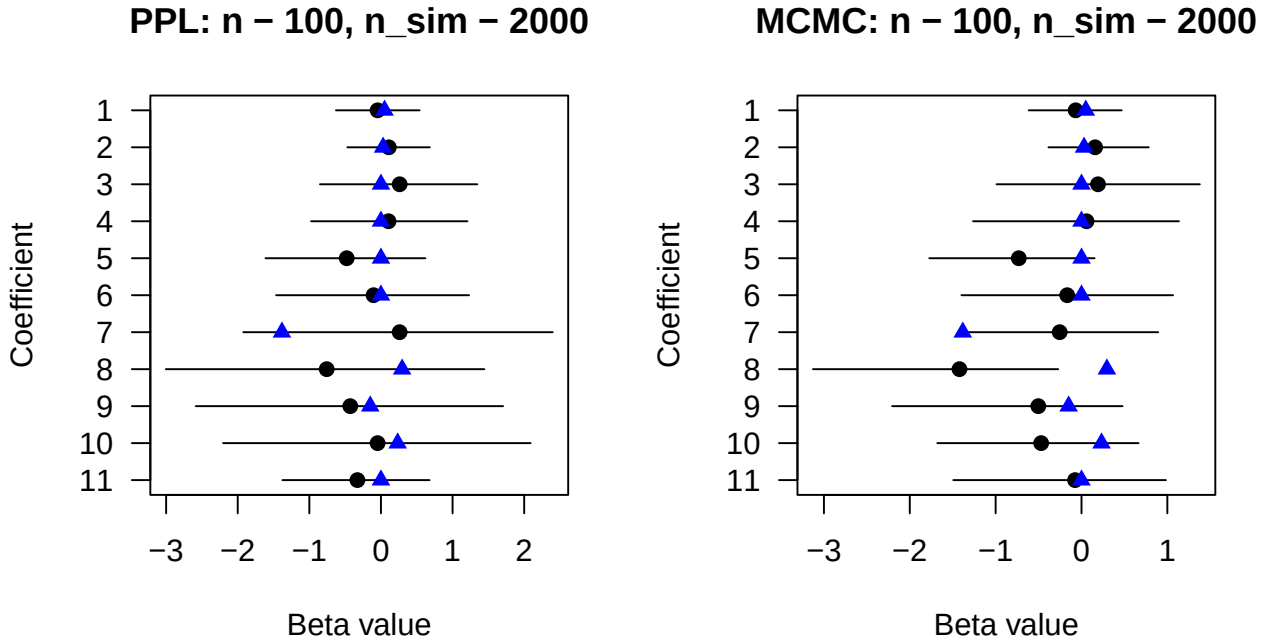


Figure 1: Coverage of each sampler. Blue Triangles: true value. Black Circle: Posterior Mean. Bars: 95% interval of the posterior estimate.

Model Fitting

We fit the model using a Metropolis-Hastings sampler, as well as a sampler in STAN using Hamiltonian Monte Carlo. The structure of our model does not allow for any conjugate priors, meaning every update step in our sampler was necessarily done using Metropolis-Hastings. The M-H sampler performed 2000 in 22 seconds while the PPL sampler performed 2000 in 48 seconds. The PPL took significantly longer than the MCMC sampler, even after extensive efforts to optimize the sampler performance. A comparison of the posterior distributions of each parameter are provided in Figure 2.

In both samplers, the posterior distributions for the continuous variables, and the factors with only 2 levels (Gender and MaritalStatus), have comparatively tight posterior distribution. The factors with multiple levels (Education and Race) have much wider distributions. However, both samplers have resulted in very similar posterior distributions for each parameters, although the MCMC sampler tended to have slightly lower parameter estimates.

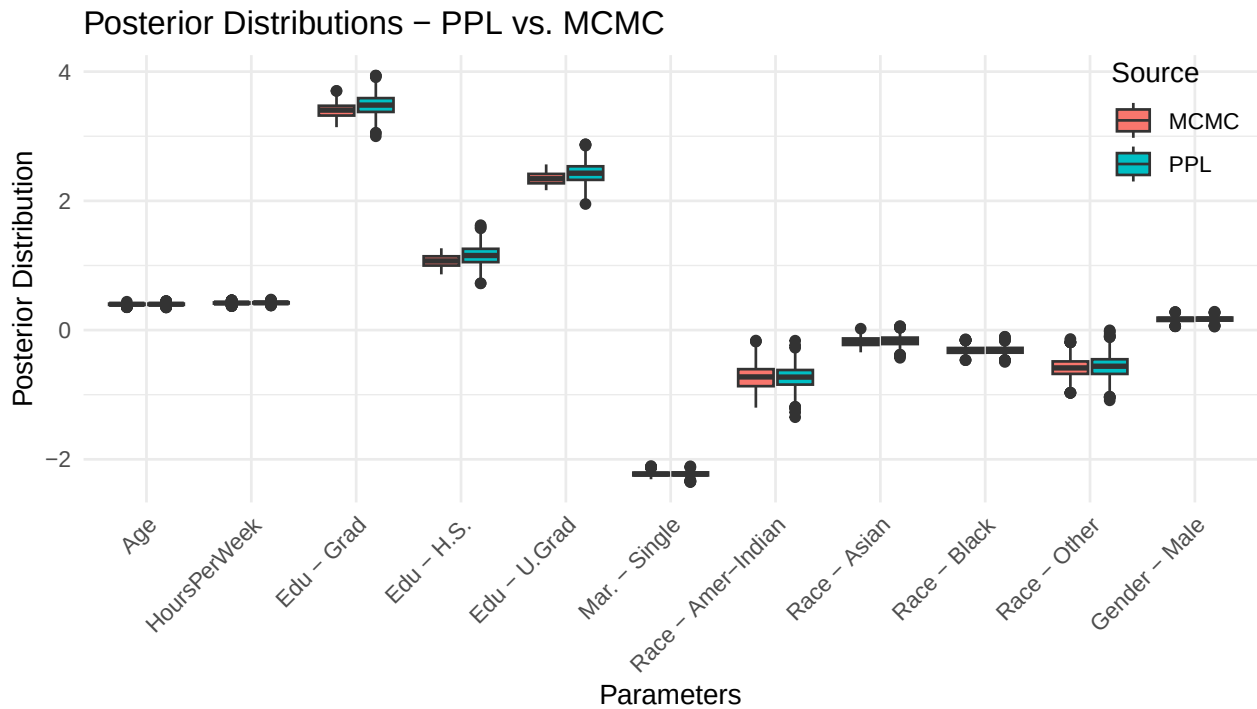


Figure 2: *Posterior distributions of each parameter. The continuous variables, and the factors with only 2 levels, are well estimated. The factors with multiple levels have much wider distributions.*

As mentioned previously, we also placed a hierarchical prior on the non-white categories of race. This is because we feel these categories should perform somewhat similarly and should have a similar population mean when compared to Race - White. To visualize the ‘sharing of strength’ between these parameters, we fit a non-bayesian linear regression to get regular parameter estimates, and compared that to the parameters estimates from our Bayesian sampling. As shown in Figure 3, the individual parameter estimates from the PPL are slightly ‘shrunk’ towards the posterior shared mean (primarily identifiable in American-Indian-Eskimo).

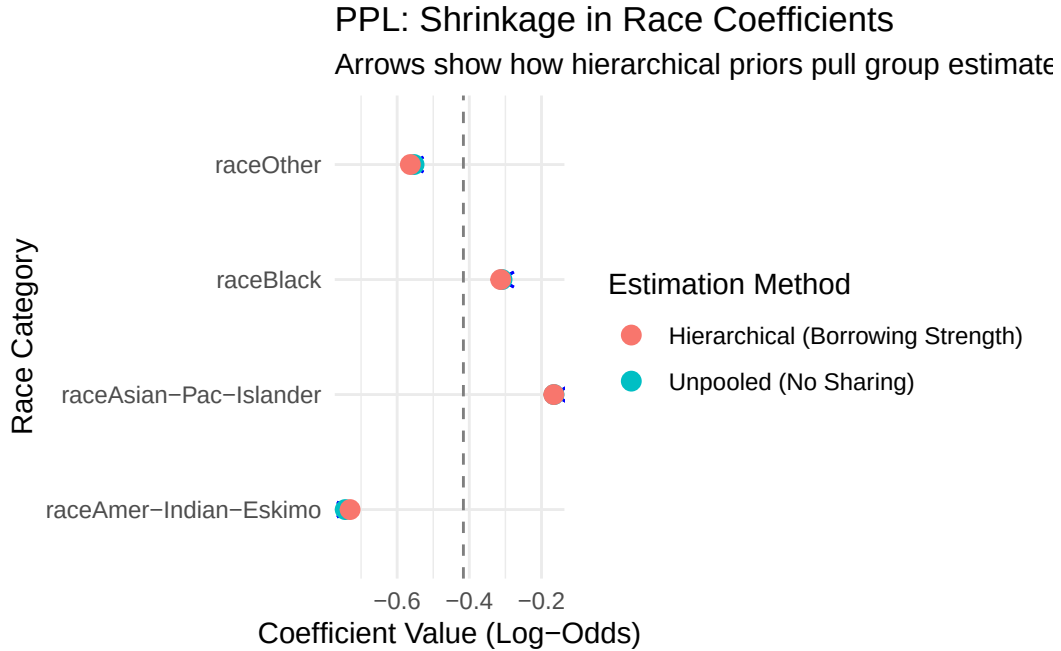


Figure 3: *Representation of the sharing of strength of the non-white race parameters. Notice that the red dots (the estimates from the PPL) are closer to the ‘true value’ when compared to the estimates from a non-bayesian linear regression. The MCMC sampler had almost identical results.*

Comparison to Non-Bayesian Analysis

To further compare the performance of the non-bayesian regression versus our Bayesian samplers, we compared the distributions of the parameter estimates. As shown in Figure 4, both methods provide very similar estimates. However, many of the parameter estimates are slightly lower in the PPL method as compared to the linear regression. The box-plots show the full posterior distribution: however, the 95% confidence interval on the mean of the posteriors (not shown) are much smaller than the 95% confidence interval on the mean from linear regression.

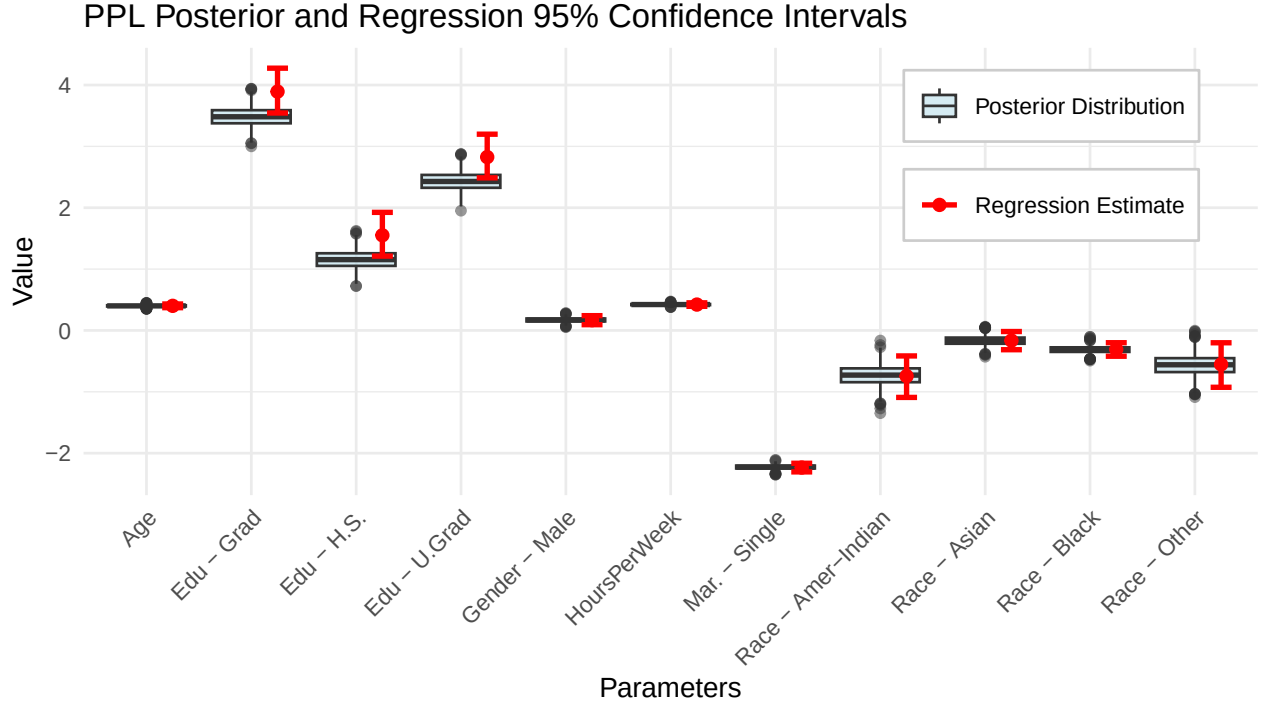


Figure 4: *Posterior distributions of the parameters vs. the 95% confidence intervals on the mean from non-bayesian linear regression.*

A few clear conclusions stand out from this plot. First, older individuals and those who work more hours in a week are significantly more likely to earn an annual salary greater than \$50k. We also notice that obtaining higher levels of education increases the likelihood of high earnings. Overall, according to our model, the individuals who are most likely to earn salaries in excess of \$50k are older, white, male, have a graduate degree, and are married. Though most of these conclusions are somewhat unsurprising, it is valuable to have more concrete evidence for these phenomena.

Sensitivity to Priors

Alternative Prior Analysis

As a sensitivity analysis, we considered an alternative prior specification that imposes stronger shrinkage on the regression coefficients. In this setting, we reduced the prior variances of the coefficients from their original values (e.g., standard deviations of 0.7 and 0.5) to a smaller common value of 0.3, and similarly reduced the intercept variance from 10 to 2. This reflects a more conservative prior belief that most predictors have relatively small effects on the response and should be shrunk toward zero unless strongly supported by the data. For the continuous predictors Age and HoursPerWeek, we retained a slight positive prior mean to reflect expected directionality, but reduced both the magnitude of the mean and the variance to limit their influence. Additionally, we strengthened the hierarchical prior on race effects by reducing the associated variance components, thereby increasing the degree of partial pooling across categories. This alternative specification

allows us to assess whether our substantive conclusions are robust to a more strongly regularizing and skeptical prior. The specific values used are provided in Table 4.

Table 4: *Table of alternate priors for sensitivity analysis*

Variable	β	Prior	Chosen Values
Intercept	0	$N(0, \sigma^2)$	$\sigma^2 = 2$
Age	1	$N(\pi, \gamma^2)$	$\pi = 0.02, \gamma = 0.3$
HoursPerWeek	2	$N(\kappa, \psi^2)$	$\kappa = 0.01, \psi = 0.3$
Education	3-5	$N(\theta, \nu^2)$	$\theta = 0, \nu = 0.3$
MaritalStatus	6	$N(\mu, \tau^2)$	$\mu = 0, \tau = 0.3$
Race	7-10	$N(\lambda, \rho^2)$	$\lambda = 0, \rho = 0.3$
Gender	11	$N(\phi, \epsilon^2)$	$\phi = 0, \epsilon = 0.3$

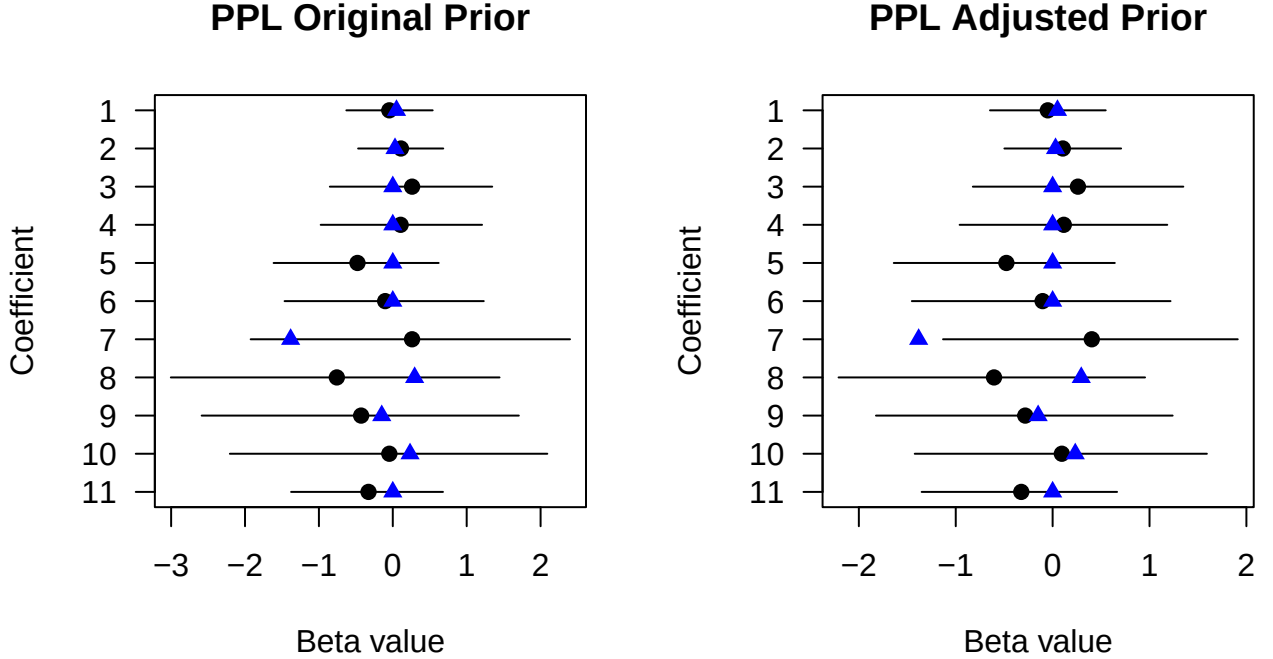


Figure 5: *Coverage of each PPL - Adjusted vs Not. Blue Triangles: true value. Black Circle: Posterior Mean. Bars: 95% interval of the posterior estimate.*

Under the stronger shrinkage prior, we observe that a subset of coefficients (notably β_3 , β_4 , β_5 , and β_{10}) remained relatively stable, with posterior means similar to those obtained under the original prior and with slightly reduced standard deviations. This suggests that these effects are strongly supported by the data and are robust to the choice of prior. In contrast, the remaining coefficients were more noticeably affected by the increased regularization, with their posterior means shrinking toward zero and their associated uncertainty increasing. This indicates that these effects are weaker and more sensitive to prior assumptions. Overall, the results suggest that only a subset of predictors have robust, well-identified effects, while others depend more heavily on the prior specification.

Summary

We used the 1994 US Census data to fit a model of the likelihood that an individual earns more than \$50k. We fit a model using a manual Metropolis-Hastings sampler, a PPL (STAN), and off-the-shelf GLM. We generated a fake dataset using ‘true’ parameters and examined the coverage of the samplers. We identified that both samplers have reasonable coverage (the PPL captured all the true betas, and the M-H missed only one). We then used both samplers on the real dataset and compared the posterior distributions for each beta. We found the distributions to be very similar, but with the PPL yielding slightly higher estimates for the mean, and slightly wider distributions.

Comparing the samplers to the GLM, we found that all the models performed relatively the same. However, the GLM has much higher Education beta estimates than either of our samplers. It also resulted in larger confidence intervals on the means of the estimates as compared to our samples. We also compared the GLM estimates for the race parameters (on which we had a hierarchical prior) to the estimates from the PPL and were able to identify a small portion of sharing of strength.

We identified that those most likely to earn salaries greater than \$50k are older, married white males who work longer hours and have a graduate degree. Though somewhat unsurprising results, both samplers and the GLM agree on these effects. These inferences are valuable to understanding an individual’s potential for higher earnings in the future.

Finally, we conducted a small study to identify sensitivity to the priors. We identified that most parameters were relatively robust to the prior values. However, some of the race parameters changed a decent amount, with one even going outside of the coverage bounds.

Appendix

Code

```
suppressPackageStartupMessages(library("rstan"))
suppressPackageStartupMessages(library("tidyverse"))
suppressPackageStartupMessages(library("ggplot2"))
suppressPackageStartupMessages(library('knitr'))
suppressPackageStartupMessages(source('functions.R'))
set.seed(1337)

data <- read_csv("train.csv")

y <- data[,15]

table_dt <- data

data <- data |>
  mutate(
    race = as.factor(race),
    age = as.numeric(scale(age)),
    hours = as.numeric(scale(`hours-per-week`)),
    education = case_when(
      education %in% c("Doctorate", "Masters") ~ "Graduate",
      education %in% c("Bachelors", "Assoc-voc", "Prof-school", "Assoc-acdm", "Some-college") ~ "College",
      education %in% c("HS-grad", "12th", "7th-8th", "9th", "10th", "11th") ~ "HighSchool",
      TRUE ~ "Elementary"
    ),
    marital = ifelse(`marital-status` %in% c("Divorced", "Separated", "Never-married", "Widowed") ~ "Married",
    )

data$race <- relevel(data$race, 'White')

X <- model.matrix(~ age + hours + education + marital + race + gender, data)[, -1]
X <- scale(X, center = TRUE, scale = FALSE)
y = data$`income_>50K`

colnames(X) <- c('age', 'hours', 'educationGraduate', 'educationHighSchool', 'educationUndergrad', 'genderMale', 'genderFemale')

p <- ncol(X)
table_dt <- table_dt |>
  mutate(
    education = case_when(
      education %in% c("Doctorate", "Masters") ~ "Graduate",
      education %in% c("Bachelors", "Assoc-voc", "Prof-school", "Assoc-acdm", "Some-college") ~ "College",
      education %in% c("HS-grad", "12th", "7th-8th", "9th", "10th", "11th") ~ "HighSchool",
    )
  )
```

```

    TRUE ~ "Elementary"
  ),
  marital = ifelse(`marital-status` %in% c("Divorced", "Separated", "Never-married", "Widowed")
)

table_dt <- table_dt[, c(1, 4, 9, 10, 13, 16)]

mySummary <- function(x) {
  c(mean = round(mean(x[x != 0]), 2), sd = round(sd(x[x != 0]), 2), q25 = round(quantile(x[x != 0], 0.25), 2), q75 = round(quantile(x[x != 0], 0.75), 2))
}

reg_summ <- t(apply(table_dt[, c('age', 'hours-per-week')], 2, mySummary))
reg_var_summ <- as.data.frame(cbind(colnames(table_dt[, c('age', 'hours-per-week')]), reg_summ))
colnames(reg_var_summ) <- c('Variable', 'Mean', 'SD', 'Q25', 'Q75')
rownames(reg_var_summ) <- NULL
kable(reg_var_summ)
table_dt <- table_dt %>% mutate(across(where(is.character), as.factor))
var_list <- c('education', 'race', 'gender', 'marital')

tot_row <- nrow(table_dt)
table <- data.frame(Variable = character(), Value = character(), Count = character(), stringsAsFactors = FALSE)
for (var in var_list){
  tab <- table(table_dt[[var]])
  tab <- cbind(var, names(tab), tab)
  tab <- tab[tab[,3] != 0, ]
  rownames(tab) <- NULL
  tab <- as.data.frame(tab)
  tab[,3] <- as.numeric(tab[,3])
  tab[, 3] <- paste0(tab[, 3], ' (', round(tab[,3]/tot_row, 3)*100.0, '%)')
  table <- rbind(table, tab)
}

colnames(table) <- c('Variable', 'Value', 'Count (Percent)')
rownames(table) <- NULL
kable(table)
alpha_var <- 10

beta_means <- rep(0, p) # Betas default to 0 mean (updated later for age and hours)
beta_sds <- rep(0.7, p) # Betas default to .7 var (updated later for age and hours)

# optional informative priors
beta_means[grep("^age", colnames(X))] <- 0.05
beta_means[grep("^hours", colnames(X))] <- 0.03

beta_sds[grep("^age", colnames(X))] <- 0.5
beta_sds[grep("^hours", colnames(X))] <- 0.5

```

```

# The non-white Race have hierarchical prior
super_p <- 1 # Number of shared parameters
shared_inds <- rep(0, p)
race_cols <- grep("^race", colnames(X))
shared_inds[race_cols] <- 1

# Hierarchical variance for race
race_shared_mean <- 0
race_var <- 0.5
race_mean_var <- 1
p <- 11
super_p <- 1
alpha_true <- -2

shared_inds <- rep(0, p)
# Race - Nonwhite
shared_inds[7:10] <- 1

n <- 100

fake_X <- generate_X(n)

beta_true <- generate_beta_true(p, shared_inds, alpha_var, prior_means = beta_means, prior_sds

fake_y <- generate_data(n, X, beta_true)
ppl_iter <- 2000
ppl_burn_in <- 500
data_stan <- generate_stan_data(n, p, fake_X, super_p, fake_y, shared_inds, alpha_var, prior_m

n_cores <- parallel::detectCores()
options(mc.cores = n_cores)

fit_stan <- stan(
  model_code = readLines("final_vec.stan"),
  data = data_stan,
  pars = c("beta"),
  iter = ppl_iter,
  warmup = ppl_burn_in,
  chains = n_cores
)
post <- rstan::extract(fit_stan)

ppl_beta_est <- apply(post$beta, 2, mean)
ppl_beta_lower <- apply(post$beta, 2, quantile, 0.025)
ppl_beta_upper <- apply(post$beta, 2, quantile, 0.975)
mcmc_iter <- 2000
mcmc_burn_in <- 500

```

```

out <- run_mcmc(X = fake_X, y = fake_y, alpha_var = alpha_var, beta_means = beta_means, beta_sds = beta_sds,
race_var = race_var, race_mean_var = race_mean_var, shared_inds = shared_inds, n_iter = mcmc_iter)

samples <- out$samples

mcmc_beta_est <- apply(samples, 2, mean)
mcmc_beta_lower <- apply(samples, 2, quantile, 0.025)
mcmc_beta_upper <- apply(samples, 2, quantile, 0.975)
par(mfrow = c(1, 2))
ord <- c(1:2,3:6,7:10,11)
ypos <- p:1

# PPL
plot(ppl_beta_est[ord], ypos, xlim = range(c(ppl_beta_lower, ppl_beta_upper)), pch = 19,
     yaxt = "n", ylab = "Coefficient", xlab = "Beta value",
     main= paste0('PPL: n - ', n, ', n_sim - ', ppl_iter))

segments(ppl_beta_lower[ord], ypos, ppl_beta_upper[ord], ypos)

axis(2, at = ypos, labels = ord, las = 2)
points(beta_true[ord], ypos, col = "blue", pch = 17)

# MCMC
plot(mcmc_beta_est[ord], ypos, xlim = range(c(mcmc_beta_lower, mcmc_beta_upper)), pch = 19,
     yaxt = "n", ylab = "Coefficient", xlab = "Beta value",
     main= paste0('MCMC: n - ', n, ', n_sim - ', mcmc_iter))

segments(mcmc_beta_lower[ord], ypos, mcmc_beta_upper[ord], ypos)

axis(2, at = ypos, labels = ord, las = 2)
points(beta_true[ord], ypos, col = "blue", pch = 17)

par(mfrow = c(1, 1))
mcmc_iter <- 2000
mcmc_burn_in <- 500

mcmc_start_time <- Sys.time()
out <- run_mcmc(X = X, y = y, alpha_var = alpha_var, beta_means = beta_means, beta_sds = beta_sds,
race_var = race_var, race_mean_var = race_mean_var, shared_inds = shared_inds, n_iter = mcmc_iter)
mcmc_end_time <- Sys.time()
mcmc_run_time <- round(as.numeric(mcmc_end_time - mcmc_start_time, units = 'secs'))

# Checking acceptance rates
# out$accept_alpha
# out$accept_beta
# out$accept_shared

```

```

samples <- out$samples

ess <- coda::effectiveSize(samples)
#discard burnin
samples <- samples[-c(1:mcmc_burn_in),]

# Posterior means
beta_hat <- colMeans(samples)

# Posterior SDs
beta_sd <- apply(samples, 2, sd)
ppl_iter <- 2000
ppl_burn_in <- 500
data_stan <- list(n = nrow(data), p = p, X = X, super_p = super_p, shared_inds = shared_inds, y = y,
                 alpha_var = alpha_var, prior_means = beta_means, prior_sds = beta_sds,
                 race_hier_mean = race_shared_mean, race_hier_var = race_mean_var, race_cols = race_cols)

params <- c("alpha", "beta", "shared_mean")

n_cores <- parallel::detectCores()
options(mc.cores = n_cores)

ppl_start_time <- Sys.time()
fit_stan <- stan(model_code = readLines("final_vec.stan"), data = data_stan, pars = params,
               iter = ppl_iter, warmup = ppl_burn_in, chains = 1)
ppl_end_time <- Sys.time()
ppl_run_time <- round(as.numeric(ppl_end_time - ppl_start_time, units = 'secs'))

sim_stan <- rstan::extract(fit_stan) # collect samples from all chains
beta_list <- c("Age", "HoursPerWeek", "Edu - Grad", "Edu - H.S.", "Edu - U.Grad", "Mar. - Single")

ppl_long <- as.data.frame(sim_stan$beta) %>% pivot_longer(cols = everything(), names_to = "Parameter")
mcmc_long <- as.data.frame(samples) %>% pivot_longer(cols = everything(), names_to = "Parameter")

# Combine datasets
combined_df <- bind_rows(ppl_long, mcmc_long)

combined_df$Parameter <- rep(beta_list, 2*(ppl_iter-ppl_burn_in))

combined_df <- combined_df %>%
  mutate(Parameter = factor(Parameter,
                           levels = beta_list))

# Create the paired boxplot
ggplot(combined_df, aes(x = Parameter, y = Value, fill = Source)) +
  geom_boxplot() +
  theme_minimal() +

```

```

  labs(title = "Posterior Distributions - PPL vs. MCMC", x = "Parameters", y = "Posterior Dist")
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust = 1), legend.position = "inside")
unpooled_fit <- glm(y ~ age + hours + education + marital + race + gender,
                   data = data, family = "binomial")
unpooled_betas <- coef(unpooled_fit)[race_cols+1]
reg_betas <- coef(unpooled_fit)[-1]
reg_betas_ci <- confint(unpooled_fit)[-1, ]
# # Extract your hierarchical posterior means from the MCMC 'samples'
# hier_betas <- colMeans(samples[, race_cols])

# # Combine into a data frame
# plot_data <- data.frame(
#   Race = colnames(X)[race_cols],
#   Unpooled = unpooled_betas,
#   Hierarchical = hier_betas
# )

# ggplot(plot_data) +
#   # Draw a vertical line at the shared mean to show the 'target' of shrinkage
#   geom_vline(xintercept = mean(sim_stan$shared_mean),
#             linetype = "dashed", color = "gray50") +
#   # Draw the connecting lines
#   geom_segment(aes(x = Unpooled, xend = Hierarchical, y = Race, yend = Race),
#             arrow = arrow(length = unit(0.2, "cm")), color = "blue") +
#   # Plot the two types of points
#   geom_point(aes(x = Unpooled, y = Race, color = "Unpooled (No Sharing)"), size = 3) +
#   geom_point(aes(x = Hierarchical, y = Race, color = "Hierarchical (Borrowing Strength)", size = 3)) +
#   labs(title = "MCMC: Shrinkage in Race Coefficients",
#        subtitle = "Arrows show how hierarchical priors pull group estimates toward the common mean",
#        x = "Coefficient Value (Log-Odds)",
#        y = "Race Category",
#        color = "Estimation Method") +
#   theme_minimal()

# Extract your hierarchical posterior means from the MCMC 'samples'
hier_betas <- colMeans(sim_stan$beta[, race_cols])

# Combine into a data frame
plot_data <- data.frame(
  Race = colnames(X)[race_cols],
  Unpooled = unpooled_betas,
  Hierarchical = hier_betas
)

ggplot(plot_data) +
  # Draw a vertical line at the shared mean to show the 'target' of shrinkage
  geom_vline(xintercept = mean(sim_stan$shared_mean),

```

```

        linetype = "dashed", color = "gray50") +
# Draw the connecting lines
geom_segment(aes(x = Unpooled, xend = Hierarchical, y = Race, yend = Race),
             arrow = arrow(length = unit(0.2, "cm")), color = "blue") +
# Draw the connecting lines
geom_segment(aes(x = Unpooled, xend = Hierarchical, y = Race, yend = Race),
             arrow = arrow(length = unit(0.2, "cm")), color = "blue") +
# Plot the two types of points
geom_point(aes(x = Unpooled, y = Race, color = "Unpooled (No Sharing)"), size = 3) +
geom_point(aes(x = Hierarchical, y = Race, color = "Hierarchical (Borrowing Strength)"), size = 3) +
labs(title = "PPL: Shrinkage in Race Coefficients",
     subtitle = "Arrows show how hierarchical priors pull group estimates toward the common mean",
     x = "Coefficient Value (Log-Odds)",
     y = "Race Category",
     color = "Estimation Method") +
theme_minimal()

beta_list <- c("Age", "HoursPerWeek", "Edu - Grad", "Edu - H.S.", "Edu - U.Grad", "Mar. - Single")

ppl_long <- as.data.frame(sim_stan$beta) %>% pivot_longer(cols = everything(), names_to = "Parameter")

ppl_long$Parameter <- rep(beta_list, (ppl_iter-ppl_burn_in))

summary_data <- data.frame(
  Parameter = beta_list,
  mean = reg_betas,
  lower = reg_betas_ci[, 1],
  upper = reg_betas_ci[, 2]
)

ggplot(ppl_long, aes(x = Parameter, y = Value)) +
# 1. Map 'fill' to a label inside aes()
geom_boxplot(aes(fill = "Posterior Distribution"), alpha = 0.5) +
# 2. Map 'color' to a label inside aes()
geom_errorbar(data = summary_data,
             aes(y = mean, ymin = lower, ymax = upper, color = "Regression Estimate"),
             width = 0.2, linewidth = 1,
             position = position_nudge(x = 0.25)) +
geom_point(data = summary_data,
          aes(y = mean, color = "Regression Estimate"),
          size = 2,
          position = position_nudge(x = 0.25)) +
# 3. Define the colors manually
scale_fill_manual(name = NULL, values = c("Posterior Distribution" = "lightblue")) +
scale_color_manual(name = NULL, values = c("Regression Estimate" = "red")) +
theme_minimal() +
theme(

```

```

    axis.text.x = element_text(angle = 45, vjust = 1, hjust = 1),
    legend.position = 'inside',
    legend.position.inside = c(0.95, 0.95),
    legend.justification = c("right", "top"),
    legend.background = element_rect(fill = "white", color = "grey80")
  ) +
  labs(title = "PPL Posterior and Regression 95% Confidence Intervals", y = "Value", x = "Parameter")
alpha_var_mod <- 2 # tighter intercept (was 10)

beta_means_mod <- rep(0, p)

# strong shrinkage toward 0
beta_sds_mod <- rep(0.3, p)

# keep slight directional belief but weaken it
beta_means_mod[grepl("^age", colnames(X))] <- 0.02
beta_means_mod[grepl("^hours", colnames(X))] <- 0.01

beta_sds_mod[grepl("^age", colnames(X))] <- 0.3
beta_sds_mod[grepl("^hours", colnames(X))] <- 0.3

# hierarchical structure stays (important-you're isolating shrinkage, not removing structure)
super_p_mod <- 1
shared_inds_mod <- rep(0, p)

race_cols_mod <- grepl("^race", colnames(X))
shared_inds_mod[race_cols_mod] <- 1

# stronger pooling across race categories
race_shared_mean_mod <- 0
race_var_mod <- 0.3
race_mean_var_mod <- 0.3
ppl_iter_mod <- 2000
ppl_burn_in_mod <- 500
data_stan_mod <- generate_stan_data(n, p, fake_X, super_p, fake_y, shared_inds, alpha_var, prior)

fit_stan_mod <- stan(
  model_code = readLines("final_vec.stan"),
  data = data_stan_mod,
  pars = c("beta"),
  iter = ppl_iter_mod,
  warmup = ppl_burn_in_mod,
  chains = n_cores
)
post_mod <- rstan::extract(fit_stan_mod)

ppl_beta_est_mod <- apply(post_mod$beta, 2, mean)

```



```

ppl_beta_lower_mod <- apply(post_mod$beta, 2, quantile, 0.025)
ppl_beta_upper_mod <- apply(post_mod$beta, 2, quantile, 0.975)
par(mfrow = c(1, 2))
ord <- c(1:2,3:6,7:10,11)
ypos <- p:1

# PPL Non-adjusted prior
plot(ppl_beta_est[ord], ypos, xlim = range(c(ppl_beta_lower, ppl_beta_upper)), pch = 19,
     yaxt = "n", ylab = "Coefficient", xlab = "Beta value",
     main= paste0('PPL Original Prior'))

segments(ppl_beta_lower[ord], ypos, ppl_beta_upper[ord], ypos)

axis(2, at = ypos, labels = ord, las = 2)
points(beta_true[ord], ypos, col = "blue", pch = 17)

# PPL Adjusted Prior
plot(ppl_beta_est_mod[ord], ypos, xlim = range(c(ppl_beta_lower_mod, ppl_beta_upper_mod)), pch = 19,
     yaxt = "n", ylab = "Coefficient", xlab = "Beta value",
     main= paste0('PPL Adjusted Prior'))

segments(ppl_beta_lower_mod[ord], ypos, ppl_beta_upper_mod[ord], ypos)

axis(2, at = ypos, labels = ord, las = 2)
points(beta_true[ord], ypos, col = "blue", pch = 17)

par(mfrow = c(1, 1))

```

```

#####
##### BY HAND FUNCTIONS #####
#####

# Log Posterior for use in MCMC
log_posterior <- function(alpha, beta, shared_params, X, y, alpha_var, beta_means, beta_sds, r

    eta <- alpha + X %*% beta
    loglik <- sum(y * eta - log1p(exp(eta)))

    logprior <- 0

    # alpha
    logprior <- logprior + dnorm(alpha, 0, sqrt(alpha_var), log = TRUE)

    # Get the prior contribution for each beta
    for (i in 1:length(beta)) {
        # race hierarchical
        if (shared_inds[i] == 1) {

```

```

    logprior <- logprior + dnorm(beta[i], shared_params[1], sqrt(race_var), log = TRUE)
  } else {
    # independent priors
    logprior <- logprior + dnorm(beta[i], beta_means[i], beta_sds[i], log = TRUE)
  }
}

# prior for race mean
logprior <- logprior + dnorm(shared_params[1], 0, sqrt(race_mean_var), log = TRUE)

return(loglik + logprior)
}

# Perform 1 MCMC step for one variable
mh_step <- function(current, proposal_sd, log_post_fn) {

  proposal <- rnorm(1, current, proposal_sd)

  log_accept_ratio <- log_post_fn(proposal) - log_post_fn(current)

  if (log(runif(1)) < log_accept_ratio) {
    return(list(value = proposal, accept = 1))
  } else {
    return(list(value = current, accept = 0))
  }
}

# Perform an MCMC simulation
run_mcmc <- function(X, y, alpha_var, beta_means, beta_sds, race_var, race_mean_var, shared_inds,
  p <- ncol(X)

  # Initialize
  alpha <- 0
  beta <- rep(0, p)
  shared_params <- 0
  proposal_sd_alpha <- 0.1
  proposal_sd_beta <- c(0.07, 0.07, 0.2, 0.1, 0.1, 0.15, 0.1, 0.1, 0.1, 0.07, 0.07)
  proposal_sd_shared <- rep(0.8, length(shared_params))

  accept_alpha <- 0
  accept_beta <- rep(0, p)
  accept_shared <- rep(0, length(shared_params))

  race_cols <- which(shared_inds == 1)
  nonrace_cols <- which(shared_inds == 0)

```

```

# Storage
beta_store <- matrix(0, n_iter, p)

for (iter in 1:n_iter) {
  # --- update alpha ---
  res <- mh_step(alpha, proposal_sd_alpha, function(a) {
    log_posterior(a, beta, shared_params, X, y, alpha_var, beta_means, beta_sds, race_var, r
  })
  alpha <- res$value
  accept_alpha <- accept_alpha + res$accept

  # --- update beta (non-race individually) ---
  for (j in nonrace_cols) {
    res <- mh_step(beta[j], proposal_sd_beta[j], function(bj) {
      beta_temp <- beta
      beta_temp[j] <- bj
      log_posterior(alpha, beta_temp, shared_params, X, y, alpha_var, beta_means, beta_sds,
    })

    beta[j] <- res$value
    accept_beta[j] <- accept_beta[j] + res$accept
  }
  # --- block update for race coefficients ---
  beta_block_current <- beta[race_cols]

  # propose new block
  proposal_block <- beta_block_current +
    rnorm(length(race_cols), 0, proposal_sd_beta[race_cols])

  # build proposed full beta
  beta_temp <- beta
  beta_temp[race_cols] <- proposal_block

  # compute log acceptance ratio
  beta_temp_lik <- log_posterior(alpha, beta_temp, shared_params, X, y, alpha_var, beta_means,
  beta_lik <- log_posterior(alpha, beta, shared_params, X, y, alpha_var, beta_means, beta_sds,
  log_accept_ratio <- beta_temp_lik - beta_lik

  # accept/reject
  if (log(runif(1)) < log_accept_ratio) {
    beta[race_cols] <- proposal_block
    accept_beta[race_cols] <- accept_beta[race_cols] + 1
  }

  # --- update shared params ---
  for (k in 1:length(shared_params)) {
    res <- mh_step(shared_params[k], proposal_sd_shared[k], function(sk) {

```

```

    sp_temp <- shared_params
    sp_temp[k] <- sk
    log_posterior(alpha, beta, sp_temp, X, y, alpha_var, beta_means, beta_sds, race_var, r
  })

  shared_params[k] <- res$value
  accept_shared[k] <- accept_shared[k] + res$accept
}

beta_store[iter, ] <- beta
}

list(samples = beta_store,
      accept_alpha = accept_alpha / n_iter,
      accept_beta = accept_beta / n_iter,
      accept_shared = accept_shared / n_iter
)
}

#####
##### DATA GENERATION #####
#####

one_hot <- function(n, k) {
  mat <- matrix(0, n, k)
  idx <- sample(1:k, n, replace = TRUE)
  mat[cbind(1:n, idx)] <- 1
  return(mat)
}

generate_X <- function(n) {

  age <- as.numeric(scale(runif(n, 18, 80)))
  hours <- as.numeric(scale(runif(n, 1, 100)))

  edu <- one_hot(n, 3)
  marital <- one_hot(n, 1)
  race <- one_hot(n, 4)
  gender <- rbinom(n, 1, 0.5)

  X <- cbind(
    age,      # 1
    hours,    # 2
    edu,      # 3-5
    marital,  # 6
    race,     # 7-10
    gender    # 11
  )

```

```

)

return(X)
}

generate_beta_true <- function(p, shared_inds, alpha_var, prior_means, prior_sds, race_hier_mean, race_hier_var) {

  # Continuous effects
  beta_true <- prior_means

  # Shared Means
  for (i in 1:p) {
    if (shared_inds[i] > 0) {
      beta_true[i] <- race_hier_mean + rnorm(1, 0, race_mean_var)
    }
  }

  return(beta_true)
}

generate_data <- function(n, X, beta_true) {

  eta <- alpha_true + X %*% beta_true
  prob <- 1 / (1 + exp(-eta))

  y <- rbinom(n, 1, prob)
  return(y)
}

generate_stan_data <- function(n, p, X, super_p, y, shared_inds, alpha_var, prior_means, prior_sds, race_hier_mean, race_hier_var, race_cols) {

  list(
    n = n,
    p = p,
    X = X,
    super_p = super_p,
    shared_inds = shared_inds,
    y = y,
    alpha_var = alpha_var,
    prior_means = beta_means,
    prior_sds = beta_sds,
    race_hier_mean = race_hier_mean,
    race_hier_var = race_mean_var,
    race_cols = race_cols
  )
}

```

GenAI Disclosure

We used GenAI to assist in troubleshooting our PPL and manual sampler. The structure of the code was written by us (ie. the function outlines, step processes, etc.), but we need a lot of help from chat in terms of optimizing the flows. Chat showed us how to code the standard-normal parameterizations in the manual sampler and PPL.